

本科科研预备练习指导书

眼在手上机械臂视觉伺服闭环控制实施手册

视觉提供误差，控制完成闭环。练习覆盖静态对准、开环对照，以及运动目标与图像延迟下的三种控制律。

文件性质：科研预备练习指导书（非正式投稿论文）

适用对象：控制专业本科生；已进入课题组、尚未承担正式课题

工作空间：以本机已有机械臂与夹爪功能包为基础

方向：视觉—控制闭环，偏重控制（稳、准、快）

文件性质：分步实施指导书（目的一原理一实现一验收）

适用对象：控制专业本科；Python 调用 MoveIt Commander；已进组、尚未承担正式课题

仿真模型：使用课设工作空间中的臂与夹爪，不使用实验室内部 URDF

语言约定：应用层用 Python；不要求编写 Commander 库或厂家 driver

下文每一步均按同一结构书写：目的（为什么做）、原理（在算什么）、实现（具体写什么、调什么）、验收（怎样才算完成）。未通过验收不得进入下一步。

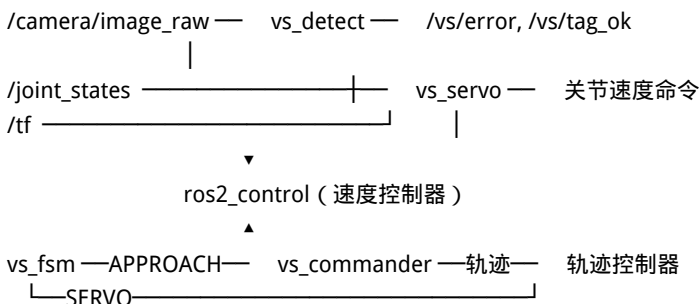
0. 项目在做什么（实施视角）

本项目要做出一条与课设“检测一次再规划”不同的链路：

- 1 用 Commander 调用 MoveIt，把末端送到相机能看见 ArUco 的位置（开环、粗、快）。
- 2 停止规划，每一帧用标签中心相对图像中心的像素误差生成速度。
- 3 速度经雅可比变成关节速度，写入 `ros2_control`，臂一动，下一帧误差就变，形成闭环。
- 4 用表格比较开环与闭环；再给图像加延迟、让标签慢速移动，比较三种控制律。

视觉只提供 $\Delta x, \Delta y$ 。论文式思考全部在控制：增益、限幅、延迟、停机。Commander 只需会调用，不必研读其源码。

节点与话题（后续步骤都挂在这张图上）：



建议新建包：vs_practice（检测、伺服、状态机、评价脚本）。规划组名、关节名以本机 SRDF 为准，下文写为 <arm_group>、<gripper_group>。

步骤 A 环境确认与信息登记

目的

避免在包名、规划组、控制器名都未知时写代码。本步不产生算法，只产生一张“接口清单”。

原理

ROS 2 里后续每个节点都要写死：规划组、关节顺序、图像话题、控制器名。这些来自已有 description / moveit_config / bringup，不是猜的。

实现

- 1 `source /opt/ros/$ROS_DISTRO/setup.bash`, 进入 `~/workspace_ws`, `colcon build` 且 `source install/setup.bash`。
- 2 启动课设 `bringup` 或 `demo` (以你仓库里实际 `launch` 为准)。
- 3 另开终端执行并抄结果:

```
ros2 topic list
ros2 interface topic /joint_states
ros2 control list_controllers
```

- 1** 打开 `SRDF/moveit_controllers.yaml`, 记下:

登记项	填入本机值
ROS 发行版	
规划组（臂）	
规划组（夹爪）	
末端 link 名	
基座 frame	
轨迹控制器名	
/joint_states 中关节顺序	

- 1 在 RViz 里用 MotionPlanning 插件拖一次末端并 Execute，确认仿真在动。

验收

清单填满；list_controllers 中轨迹控制器为 active；RViz 执行一次成功。不通过则只排启动问题，不写业务节点。

步骤 B 用 Python 调用 Commander 完成开环流程（第 1 关）

目的

把第八章的“会调用”落成状态机骨架：以后视觉只替换“目标从哪来”，不重写接近逻辑。

原理

Commander 是 MoveIt 的客户端。set_pose_target 只表达“希望末端到哪”；plan() 在关节空间搜索无碰路径；execute() 把 JointTrajectory 交给轨迹控制器。调用者不等价于驱动器。笛卡尔直线 compute_cartesian_path 在笛卡尔空间插路点，再 Inverse 成关节轨迹，仍由同一控制器跟踪。夹爪通常是另一规划组或 GripperCommand，与臂的 6/7 个关节分开。

状态：APPROACH → DESCEND → GRASP → LIFT → DONE。任一步 `error_code != SUCCESS` 则 ERROR 并停止。

实现

1 在 RViz 拖出两个安全位姿，抄到参数文件（单位 m、rad）：

```
# vs_practice/config/waypoints.yaml
base_frame: base_link
approach:
  position: {x: 0.30, y: 0.00, z: 0.35}
  orientation: {x: 0.0, y: 1.0, z: 0.0, w: 0.0} # 按本机末端朝下改
descend_dz: -0.08 # 相对接近点沿工具 Z 或基座 Z 下降
```

姿态四元数必须在 RViz 里试过，禁止抄手册数字直接用于你的臂。

1 新建 vs_commander_node.py（API 名称跟第八章走，下列为常见写法）：

```
# 伪代码：以课程第八章实际类名为准（MoveGroupCommander / MoveItPy / pymoveit2）
arm = create_move_group("<arm_group>")
arm.set_pose_target(approach_pose)
ok = arm.go(wait=True)
arm.stop()
arm.clear_pose_targets()

# 下降：笛卡尔路点
waypoints = [current_pose, lowered_pose]
plan, fraction = arm.compute_cartesian_path(waypoints, eef_step=0.01)
if fraction < 0.95:
    raise RuntimeError(f"cartesian fraction={fraction}")
arm.execute(plan, wait=True)

gripper = create_move_group("<gripper_group>")
# 或发布 GripperCommand；以课设为准
close_gripper(gripper)
```

1 每步打印 state 与返回码。用参数 ~waypoints 读 yaml，不要把数字写死在多处。

2 录包：ros2 bag record /joint_states /tf。

3 连续自动运行两遍，中间不拖 RViz。

验收

- 不打开 MotionPlanning 手动目标，脚本能走完全流程。
- `fraction >= 0.95`，夹爪有开合（若本组无夹爪，可省略 GRASP，在笔记声明）。
- 能向自己复述：谁规划、谁跟踪轨迹。

常见失败：规划失败——接近点在碰撞或超限，改 yaml 而不是改 MoveIt

源码。笛卡尔失败——下降距离减半。臂不动——回到步骤 A 看控制器是否 active。

步骤 C 眼在手相机与 ArUco 检测（第 2 关前半）

目的

让图像成为系统输入。本步只做测量，不做控制。

原理

针孔模型：空间点投影到像素 $((u,v))$ 。ArUco

在图像上提供四个角点，中心取平均作为特征。眼在手表示相机固连末端，手眼为常值

$(T_{\mathrm{ee}}^{\mathrm{cam}})$ 。仿真中该变换写在 URDF 的 camera_link 相对法兰的 $\langle \text{origin} \rangle$ ，与 TF 必须一致。光学坐标系通常是 (Z) 沿光轴向前、 (X) 向右、 (Y) 向下，与机械法兰系不同，连乘时不得混用。

实现

1 挂相机（仿真）。在末端 link 下增加 camera_link 与 camera_optical_frame。Gazebo 相机插件发布：

- /camera/image_raw (sensor_msgs/Image)
- /camera/camera_info (内参)

用 ros2 topic hz /camera/image_raw 确认帧率（建议 20 ~ 30 Hz）。

1 放标签。世界中固定 ArUco 模型或贴图。参数：

```
aruco:
  dictionary: DICT_4X4_50
  id: 0
  marker_length_m: 0.05
```

1 检测节点 vs_detect。订阅图像，OpenCV aruco.detectMarkers，发布：

话题	类型	内容
/vs/tag_ok	std_msgs/Bool	本帧是否看见
/vs/pixel	geometry_msgs/Point	$((u,v))$ 中心， (z) 不用
/vs/error	geometry_msgs/Vector3	$(e_u=u-u^*)$ ， $(e_v=v-v^*)$ （可先发 0，伺服步再算）

图像上画中心十字、检测框，用 image_view 或 OpenCV 窗口确认。

1 TF 核对。ros2 run tf2_ros tf2_echo <ee_link> camera_optical_frame，与 URDF origin 一致。

验收

看见标签时 /vs/tag_ok 为真；遮挡后为假；像素随标签平移而变。本步 禁止 把像素送给 MoveIt。

步骤 D 开环视觉引导（第 2 关后半）

目的

证明“看见”可以生成一个位姿目标，同时留下开环基线：运动中途不更新目标。

原理

若检测给出相机系中标签位姿 $(T_{\mathrm{cam}}^{\mathrm{tag}})$ ，则

$($

$T_{\{\mathrm{base}\}}^{\{\mathrm{tag}\}} = T_{\{\mathrm{base}\}}^{\{\mathrm{ee}\}} T_{\{\mathrm{ee}\}}^{\{\mathrm{cam}\}} T_{\{\mathrm{cam}\}}^{\{\mathrm{tag}\}}$.

$\backslash$

预对准位姿取标签前方、沿相机光轴后退 $\backslash(d)$ (例如 0.20 m), 姿态使相机对着标签。该式只在 APPROACH 开始时算一次, 然后交给步骤 B 的 Commander。执行过程中标签再动, 轨迹不变, 故为开环。

仅有像素没有三维位姿时: 本关可用“写死深度 + 像素近似横向偏移”估一个粗目标, 精度差但足够对比; 第 3 关改用纯像素误差, 不再依赖这次估的位姿。

实现

- 1 在 vs_fsm 的 APPROACH 入口: 等待 /vs/tag_ok, 取一次位姿, 计算 approach_pose, 调用步骤 B 的 go()。
- 2 日志打印: 检测时刻的 $\backslash(u,v)$ 与目标位置。
- 3 执行完毕后再读一次 $\backslash(u,v)$, 记为开环终值误差 (供步骤 F)。
- 4 手动平移标签, 重新运行, 确认新目标变化; 单次执行中途不重新 plan()。

验收

能说明: 图像只用了一次。保存 bag。不得在本关把误差送进速度环。

步骤 E 静态视觉速度闭环 (第 3 关, 核心)

目的

把控制律接到执行层。这是本项目与课设抓取的分界。

原理

误差。参考像素取图像中心 $\backslash((u^*, v^*) = (c_x, c_y))$ (来自 camera_info)。 $\backslash(e = [e_u, e_v]^{\mathrm{T}})$ 。

图像速度 (简化 IBVS)。先做二维对中 (深度变化暂不作为主通道):

$\backslash$

$$v_x = -\lambda e_u, \quad v_y = -\lambda e_v, \quad \omega = 0$$

$\backslash$

再按光学系定义把 $\backslash((v_x, v_y))$ 映到相机系线速度 (注意 $\backslash(u)$ 增大常对应光学系 $\backslash(X)$, 须用实机/仿真试符号)。更完整的 IBVS 使用交互矩阵 $\backslash(L(u, v, Z))$, 第 3 关可用常值深度 $\backslash(\hat{Z})$ 近似:

$\backslash$

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

```

= -\lambda \hat{Z}

\begin{bmatrix} 1/f_x & 0 \\ 0 & 1/f_y \end{bmatrix}

\begin{bmatrix} e_u \\ e_v \end{bmatrix}.

\]

```

关节速度。取相机系或末端系速度旋量 $\mathbf{V}$ (未用的分量置 0),

```

\|

\dot{\mathbf{q}} = \mathbf{J}^+ \mathbf{V}, \quad

\mathbf{J}^+ = \mathbf{J}^{\mathrm{T}} (\mathbf{J}^{\mathrm{T}} \mathbf{J} + \mu \mathbf{I})^{-1}.

\]

```

μ 在近奇异时增大。 $\dot{\mathbf{q}}$ 必须限幅。

离散闭环。周期 Δt (建议与图像同级, 20 ~ 50 Hz)。图像延迟一帧时, 律仍用旧 $\mathbf{e}$ 积分速度, 等效额外开环, λ 过大即振荡。这是第 5 关的伏笔, 第 3 关先把无延迟情况调稳。

停机。 $|\mathbf{e}| < \epsilon$ 连续 N 周期 $\rightarrow$ HOLD (速度清零)。tag_ok 连续丢失 M 帧 $\rightarrow$ ABORT。 $|\dot{\mathbf{q}}|$ 超限 $\rightarrow$ 先饱和, 仍超则 ABORT。

实现

- 1 控制器。步骤 B 用的是轨迹控制器。伺服前切换到速度控制器 (名称以本机为准, 常见如 joint_group_velocity_controller 或转发速度控制器)。

```

ros2 control switch_controllers \
--activate joint_group_velocity_controller \
--deactivate <trajectory_controller>

```

若硬件/仿真没有速度接口: 先查官方是否支持; 不要用 2 Hz 位置点冒充闭环。接口确认写入步骤 A 清单。

- 1 雅可比。用 URDF + robot_state 计算 (ikpy 仅作下策)。关节顺序必须与速度命令一致。
- 1 节点 vs_servo (50 Hz 定时器):

```

if not tag_ok:
    lost += 1
    publish_zero_velocity()
    if lost > M: fsm.abort()
    return
lost = 0
e = pixel - image_center
V_cam = image_law(e, lam, Z_hat, fx, fy) # 只改这一函数, 第 5 关换律
V_base = transform_twist(V_cam, T_base_cam) # 若 J 在基座系
qdot = damped_pinv(J, V_base, mu)
qdot = np.clip(qdot, -qdot_max, qdot_max)
publish_velocity(qdot)

```

- 1 状态机 vs_fsm：APPROACH（步骤 D 或写死接近）成功后 switch 到速度控制器，再 SERVO。进入 HOLD/ABORT 后切回轨迹控制器，便于下次规划。
- 1 调参顺序： λ 从很小开始（刚能蠕动）→ 增大至出现振荡 → 回调 20% ~ 30%。先调限幅再调 λ 。只改参数，不改检测器。
- 1 初始偏置实验：标签在画面左、右、上各一次，记录能否对中。

验收

- 伺服段日志中 零次 plan()。
 - 误差曲线收敛或阻尼收敛，反向则先改符号再改 λ 。
 - 遮挡标签后速度变 0。
 - 能书面写出： e 在哪个坐标系、 V 在哪个坐标系、 J 的行对应哪些关节。
- 本步是进组演示最低完成线。

步骤 F 开环 / 闭环对照（第 4 关）

目的

把“稳、准、快”变成可重复数字，而不是视频观感。

原理

开环：一次视觉 + 一次规划，终值误差含标定与规划公差，过程中无反馈。闭环：过程中减误差，终值由 ϵ 、噪声和增益决定； λ 大则快但可能振荡。公平比较要求同一初始关节、同一标签、同一相机。

实现

- 1 参数 mode:=open|closed。开环跑步骤 D 并在到位后采 1 s 误差；闭环跑步骤 E。
- 2 每模式 ≥5 次。失败定义：超时 $T_{\max}$ 、丢失、超限、碰撞。失败样本保留。
- 3 由 bag 计算：

指标	计算		
准	最后 0.5 s 的 mean	e	λ （像素）
快	首次进入 ϵ 并保持 0.3 s 的时间		
稳	λ	e	λ 过零或峰值次数；最大超调

- 1 出表与 $|e|(t)$ 曲线，禁止目视填数。

验收

有表、有曲线、有一句结论（例如闭环更准但某次因 λ 偏大振荡）。缺表视为本关未完成。

步骤 G 运动目标、延迟与三种律（第 5 关）

目的

在步骤 E 已稳定的环上增加时变目标与过时反馈，迫使思考离散控制，而不是再做一个检测器。

原理

检测与控制之间插入延迟 T_d 后，律使用 $e(t-T_d)$ 。固定 λ 仍按“当前还差很多”加油门，相位滞后导致振荡。降增益 $\lambda(T_d)=\lambda_0/(1+T_d/\tau)$ 减小开环增益，稳而可能慢。一拍预测 $\hat{e}=e+\dot{e}T_d$ （ $\dot{e}$ 可用差分加低通），试图对齐相位，模型错则超调。三种律必须共享测量、限幅、停机，只替换 `image_law()`。

实现

- 1 标签沿直线慢速往复（Gazebo 关节或脚本改 pose），速度先取 1~2 cm/s。
- 2 `vs_detect` 后增加缓冲队列，按 `image_delay_ms` 输出过时 `/vs/error`。
- 3 参数 `law:=p|gain_schedule|predict`，只改伺服中的一处函数。
- 4 实验格：静止 $T_d=0$ ；静止 $T_d \in \{50, 100, 150\}$ ms；慢速运动 + 同一组延迟。每格重复多次。
- 5 记录跟踪误差均值/峰值、振荡、失败率。写清预测在哪一档延迟变差。

验收

launch 能复现全部参数；能用“过时误差 + 增益”解释表格。未完成步骤 E 不得进入本步。

步骤 H 真机（第 6 关，可选）

目的

在真实延迟与手眼误差下复现步骤 E/F。不更换算法主题。

原理

真机与仿真的差异主要是：手眼不准、图像与关节不同步、摩擦与限速。控制结构不变：接近仍调用 `Commander`，伺服仍走速度接口。实验室用官方 ROS 包，不抄课设 hardware 源码。

实现

- 1 导师确认：臂的 ROS 版本、驱动包名、法兰能否装相机、急停规程。
- 2 官方例程完成使能与 TCP 慢速直线运动。
- 3 打印标签，标定手眼，记录残差。
- 4 限速为仿真的保守值，复现步骤 E，再视时间做 F。步骤 G 可用手缓慢挪标签，延迟仍用软件。

5 全程 bag；结束松使能。调参回仿真，不在臂上盲加 λ 。

验收

真机能对中或能解释失败来自标定/接口而非逻辑抄错。占机保持短时、集中。

进度、依赖与当前状态

- A 清单 → B 调用 Commander → C 检测 → D 开环视觉
 - E 速度闭环 ★最低完成线
 - F 对照表
 - G 延迟与三种律 ★含金量
 - H 真机

步骤	状态
A 环境清单	未开始（第八章完成后立即做）
B Commander 流程	未开始
C 检测	未开始
D 开环视觉	未开始
E 静态闭环	未开始
F 对照	未开始
G 运动+延迟	未开始
H 真机	未开始

课程安排：第一门第八章 Python 调用 → 步骤 B。第二门仅在速度控制器起不来时补 Mock 配置。不写 C++ Commander，不写厂家 driver。

向协作助手汇报时请附：当前步骤、改动的节点、终端关键输出、成功或失败现象。